

Live Coding Drills — Spring Boot (GYG tarzı)

Kaynak sentezi: arkadaş notları + Medium Senior Java guide + Reddit Spring bug-fix + resmi interview repo README indeksi (JDK 21, Gradle, :8080).

0. Ortam checklist

- ☐ JDK 21
- ☐ IntelliJ (debug)
- ☐ Gradle wrapper
- ☐ Uygulama `http://localhost:8080`
- ☐ HTTP client (IntelliJ HTTP, curl, Postman)
- ☐ Timer (75 dk)
- ☐ (Opsiyonel) ikinci kişi mock interviewer

Resmi repo indeksi (private olabilir):

<https://github.com/getyourguide/swe-be-coding-interview>

1. Interviewer ne bakıyor? (Medium feedback'ten)

İyi

- Thought process
- Log ile satıra inme
- Test yazma refleksi
- Merak / soru sorma
- Guidance ile ilerleme (ama senior'da az bağımlılık)

Kötü

- Root-cause kaçırma
 - Yarım test
 - Time management çöküşü
 - Refactor'u sadece verbal
-

2. Drill A — Bug fix only (30 dk)

Setup (kendi projende bilerek boz)

`BookingService.create :`

- Supplier null olduğunda NPE
- Veya wrong HTTP 200 on validation error

- Veya exception swallow + empty list

Script

1. Fail eden request'i göster
2. Log/stack
3. "Null check eklerdim" demeden **neden null** diye sor
4. Fix
5. Test: null supplier, happy path

Pass kriteri

- 15 dk içinde doğru root-cause
 - En az 2 test
 - Sesli narration
-

3. Drill B — Feature endpoint (30 dk)

Prompt

"POST /api/v1/tours/{tourId}/reviews — rating 1–5, comment optional, author required.
Invalid rating → 400. Tour yok → 404."

Beklenen parçalar

- DTO + validation (@Min @Max @NotBlank)
- Service rule
- Repo
- Controller
- Exception handler mapping
- Unit test service + MockMvc/WebTestClient smoke

Senior plus

- Idempotency?
 - Auth placeholder?
 - Pagination later?
-

4. Drill C — Refactor (30 dk)

Prompt

God class: controller içinde business + DB + mapping.

Hedef

- Controller thin
- Service cohesive
- Private methods / value objects
- Duplicate validation tek yerde
- Testable seams

Sesli cümleler

“Bu mapping controller’da kalmamalı; service’e taşıyorum ki HTTP’den bağımsız test edeyim.”

“Exception’ı generic Runtime’a çevirmek yerine domain exception + handler.”

5. Drill D — Full mock (75 dk) — asıl prova

Senaryo paketi (kendine ver)

1. **Bug:** GET /activities/{id}/availability wrong empty when supplier inactive vs null
2. **Feature:** POST /activities/{id}/holds
3. **Refactor:** availability calculation service’e + cache field

Zaman kutuları

Bkz. cheatsheet.

Rubrik (kendine 1-5)

Kriter	Skor
Root-cause doğruluğu	
Test kalitesi	
Feature tamamlanma	
Refactor somutluğu	
İletişim / zaman	
Prod düşüncesi (metrics, edge)	

Hedef toplam $\geq 24/30$

6. Layered bug fikirleri (Medium: her katmanda issue)

Katman	Örnek bug
Entity	Yanlış enum default; missing @Version

Repository	Wrong query method name; ignore case miss
Service	Off-by-one capacity; timezone
Controller	Wrong status code; ignoring validation
Exception handler	Catch-all hiding 404 as 500
Tests	Happy-path only; no failure case
Config	Wrong property → null bean

7. AI kullanım drill

Aynı Drill D'yi bir kez **AI ile**, bir kez **AI'sız** yap.

Not et:

- Nerede hızlandı?
- Nerede yanlış yönlendirdi?
- Interviewer'a nasıl izah ederdin?

Reddit notu: AI yavaşlatabiliyor — kör paste yapma.

8. Java sohbet köprüsü (coding bitince gelebilir)

Hazır 60 sn:

"Bu hold servisinde allocation açısından dikkatim: her request'te büyük DTO graph map etmemek, log'larda string concat patlatmamak. Prod'da JFR ile allocation profile alır, G1 pause p99'a bakarım. -Xms/-Xmx eşitleyip gc log'larını dashboard'a koyarım."

9. Junior farkı (bilgi için)

Associate thread: Vue/JS codebase share + bugs.

Sen Senior'sun → **Java/Spring derinliği** + refactor ownership.

Ama format aynı: **mevcut kodda çalış**.

Android public challenge (pattern referansı):

<https://github.com/getyourguide/android-challenge>

10. Günlük mini rutin (7 gün)

Gün	Drill
-----	-------

1	A
2	B
3	C
4	D
5	A+B hız
6	D (AI policy ile)
7	D final + retrospektif not

Retrospektif soruları:

1. En çok zamanı nerede yedim?
2. İlk hipotezim doğru muydu?
3. Test'i ne zaman yazdım?
4. Refactor somut muydu?